1. Why LangGraph exists?
2. What is LangGraph?
3. LangChain vs LangGraph (When to use what)

Important:

1. Video will be unapologetically long
2. Prerequisite

What is LangChain

LangChain is an open-source library designed to simplify the process of building LLM based applications.

It provides modular building blocks that let you create sophisticated LLM-based workflows with ease.

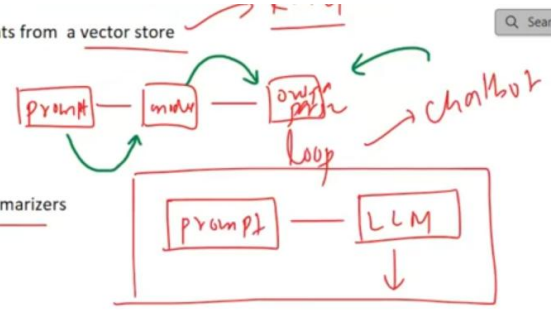
LangChain consists of multiple components

1. **Model** components gives use a unified interface to interact with various LLM providers
2. **Prompts** component helps you engineer prompts
3. **Retrievers** component helps you fetch relevant documents from a vector store

But the biggest offering of LangChain is **Chains.**

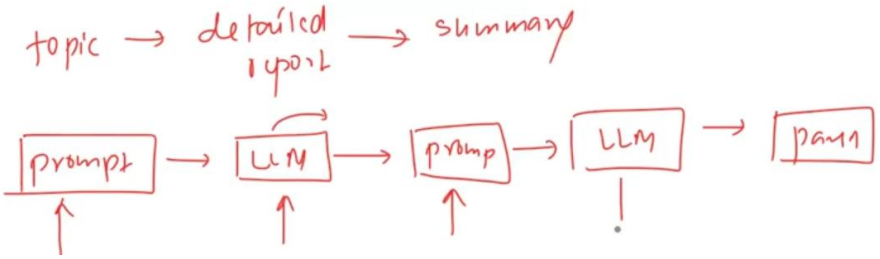
3. **Retrievers** component helps you fetch relevant documents from a vector store

But the biggest offering of LangChain is **Chains.**



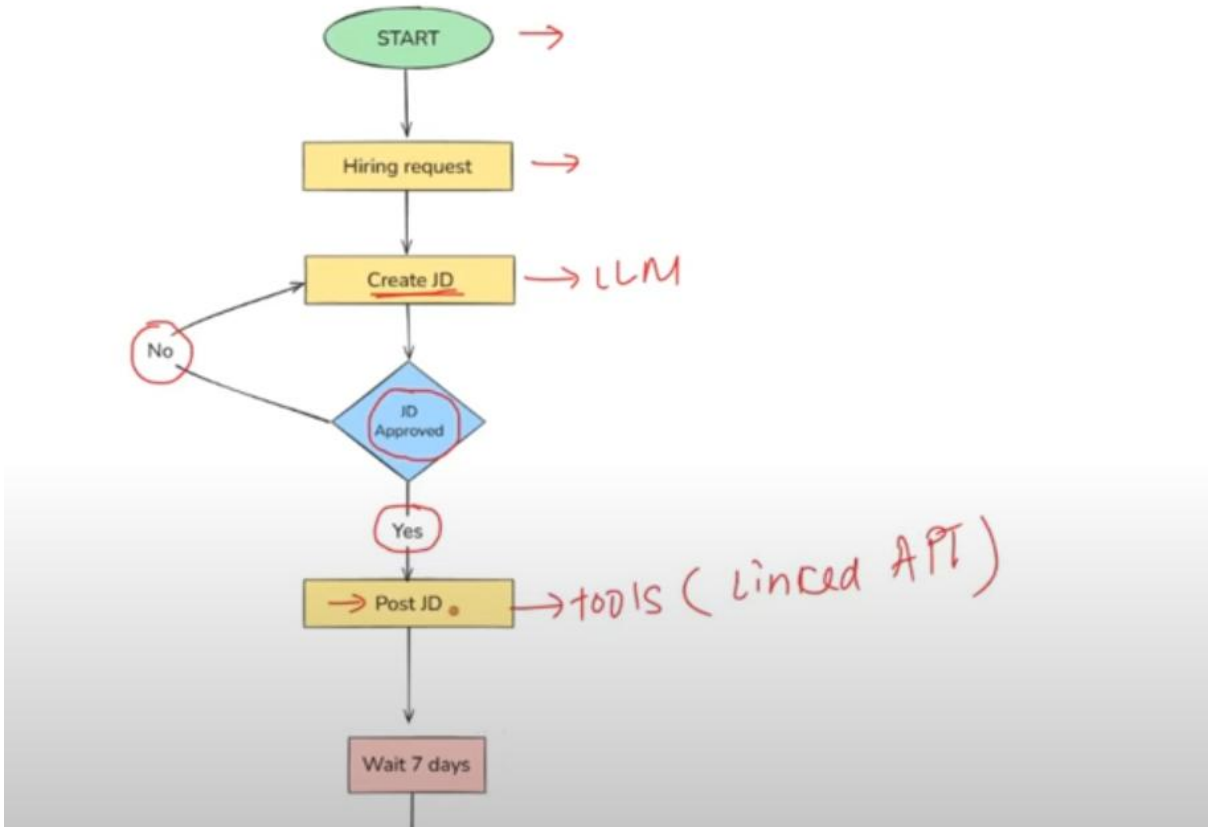
What can you do with LangChain

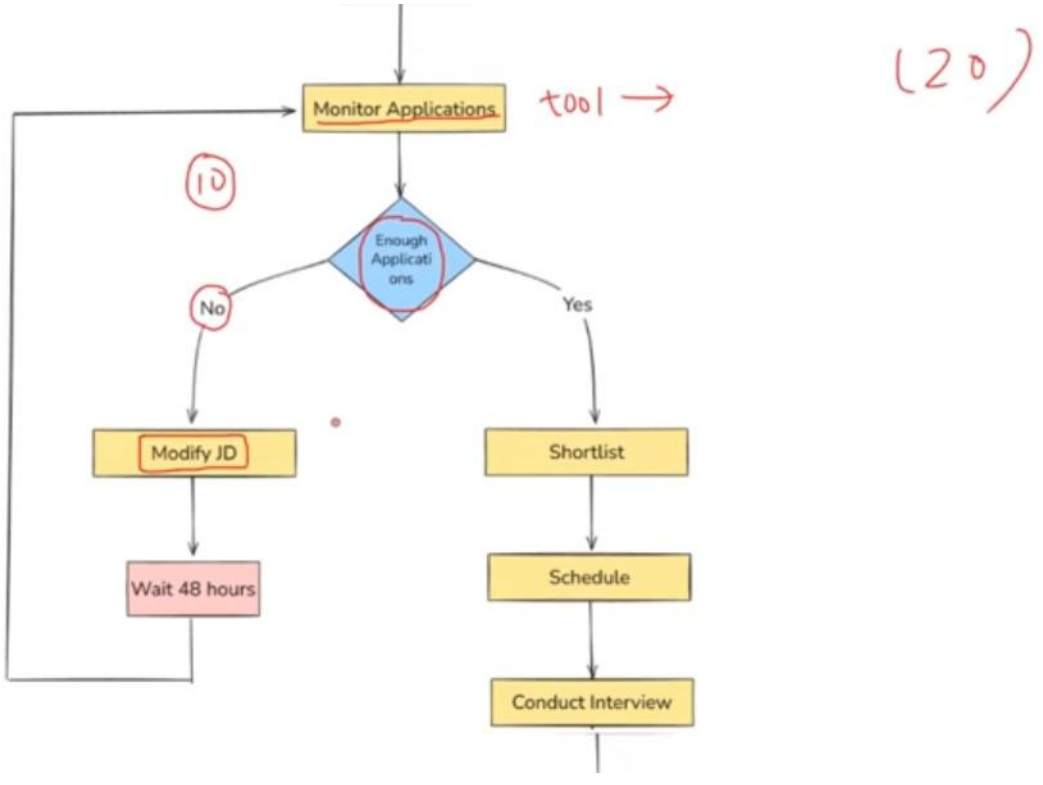
1. Simple conversational workflows like Chatbots, Text Summarizers
2. Multistep workflows
3. RAG applications
4. Basic level agents

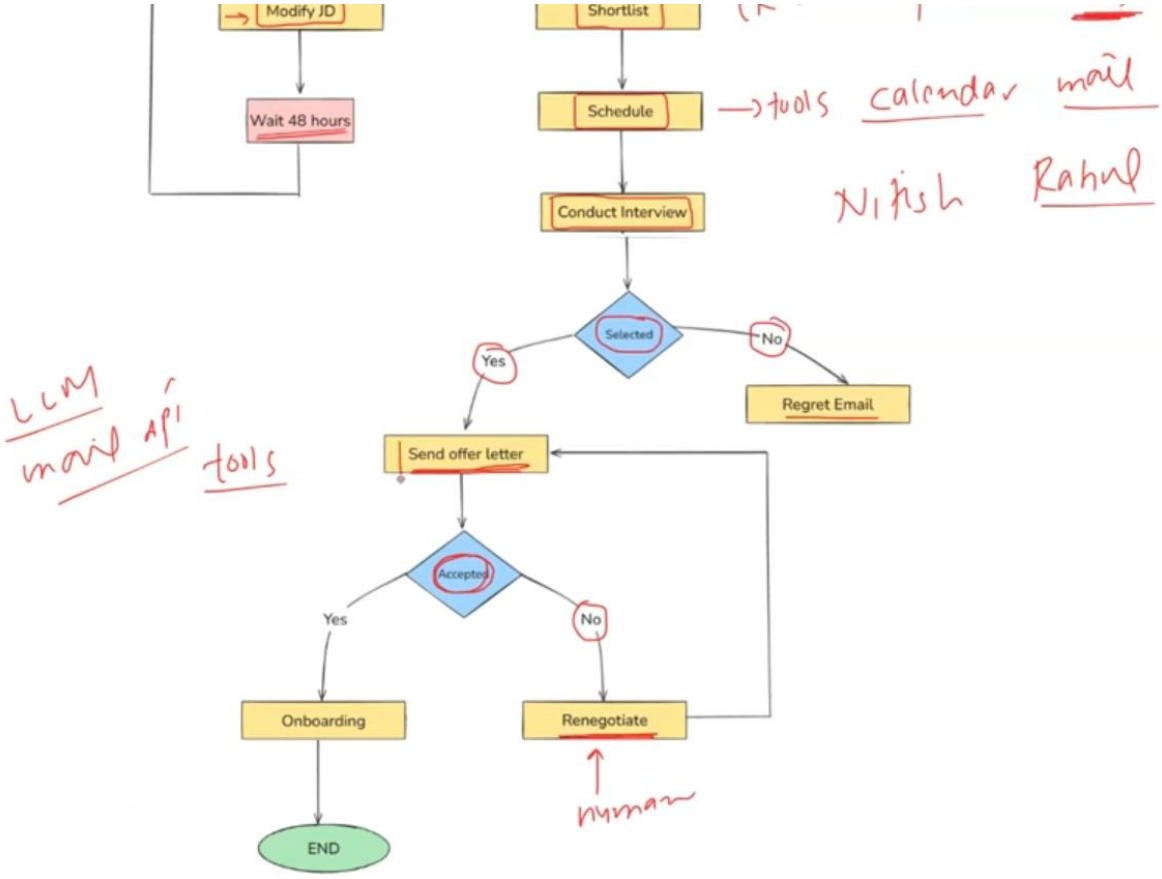


Automated Hiring workflow

03 July 2025 09:15







Challenges

03 July 2025 07:59

1. Challenge #1 - Control Flow Complexity

1) conditional branch
2) loops
3) jump

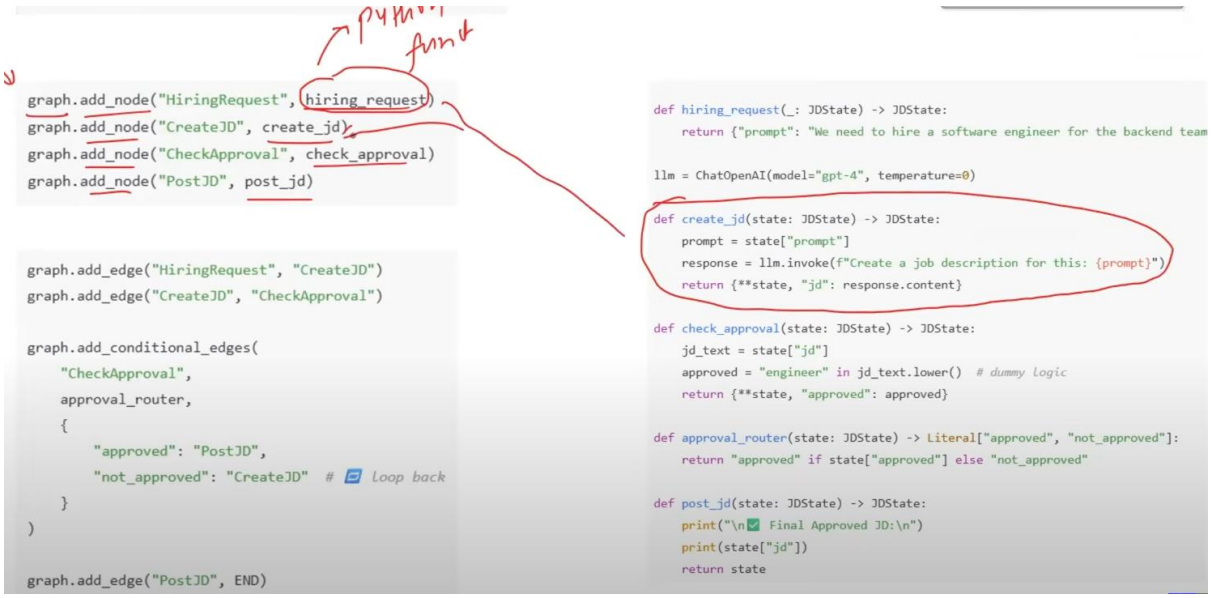
```
hiring_prompt = "We need to hire a Software Engineer for our backend team."
```

```
# Step 2: Create JD using LLM
llm = ChatOpenAI(model="gpt-4", temperature=0)

jd_prompt = ChatPromptTemplate.from_template(
    "Create a job description based on the hiring request:\n\n{request}"
)
jd_chain = jd_prompt | llm | StrOutputParser()
```

```
# Step 3: Approval function
def approve_jd(jd: str) -> bool:
    # Simulate approval logic
    return "Approved"
```

```
# Step 4: Post JD function
def post_jd(jd: str):
    print("JD Approved and Posted:\n")
```

```
# Step 5: Loop until JD is approved
approved = False
jd_output = None

while not approved:
    jd_output = jd_chain.invoke({"request": hiring_prompt})
    print(jd_output)

    approved = approve_jd(jd_output)

    if not approved:
        print("JD not approved. Regenerating...\n")

# Final Step
if approved:
    post_jd(jd_output)
```



```python
graph.add_node("HiringRequest", hiring_request)
graph.add_node("CreateJD", create_jd)
graph.add_node("CheckApproval", check_approval)
graph.add_node("PostJD", post_jd)

graph.add_edge("HiringRequest", "CreateJD")
graph.add_edge("CreateJD", "CheckApproval")

graph.add_conditional_edges(
    "CheckApproval",
    approval_router,
    {
        "approved": "PostJD",
        "not_approved": "CreateJD"  # 🔁 Loop back
    }
)

graph.add_edge("PostJD", END)
```

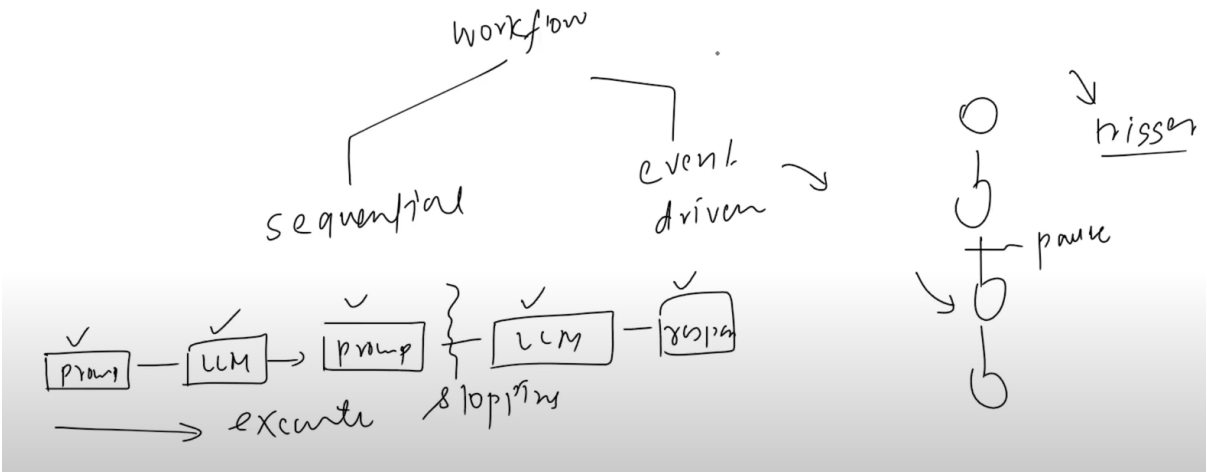
```python
def hiring_request(_: JDState) -> JDState:
    return {"prompt": "We need to hire a software engineer for the backend team."}

llm = ChatOpenAI(model="gpt-4", temperature=0)

def create_jd(state: JDState) -> JDState:
    prompt = state["prompt"]
    response = llm.invoke(f"Create a job description for this: {prompt}")
    return {**state, "jd": response.content}

def check_approval(state: JDState) -> JDState:
    jd_text = state["jd"]
    approved = "engineer" in jd_text.lower()  # dummy logic
    return {**state, "approved": approved}

def approval_router(state: JDState) -> Literal["approved", "not_approved"]:
    return "approved" if state["approved"] else "not_approved"

def post_jd(state: JDState) -> JDState:
    print("\n✅ Final Approved JD:\n")
    print(state["jd"])
    return state
```

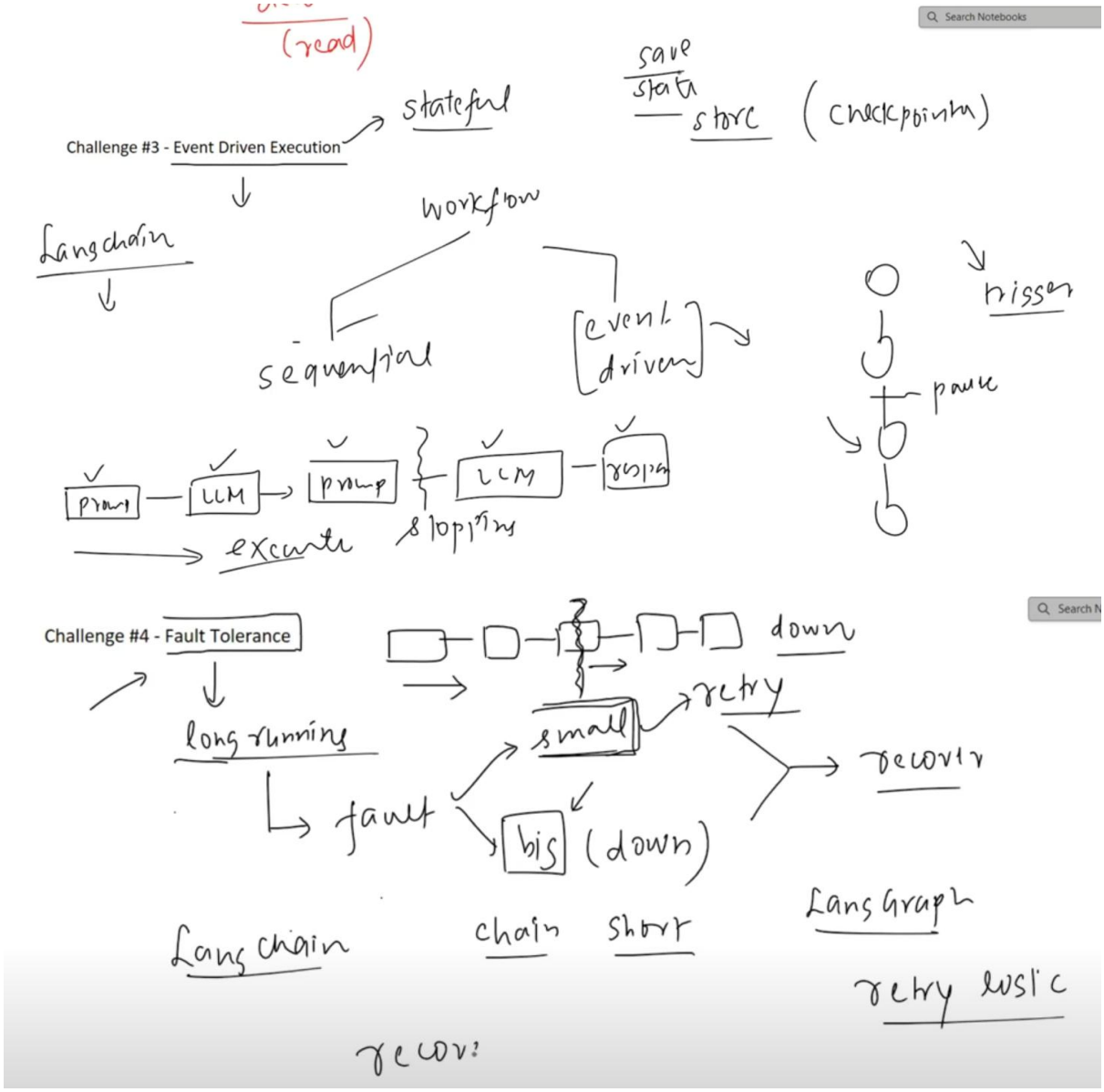
Challenge #2 - Handling State

```
state = {
    "goal": "Hire a backend software engineer",
    "jd": "",  # Job description text
    "jd_approved": False,
    "jd_posted": False,
    "min_applicants": 5,
    "num_applications": 0,
    "shortlisted_candidates": {
        # "candidate_id": {"name": ..., "score": ..., "status": ...}
    },
    "interview_questions": [],
    "offer_status": {
        "sent": False,
        "accepted": False,
        "renegotiated": False
    },
    "onboarding_status": {
        "completed": False,
        "start_date": None,
        "employee_id": None
    }
}
```

Challenge #3 - Event Driven Execution

Challenge #3 - Event Driven Execution



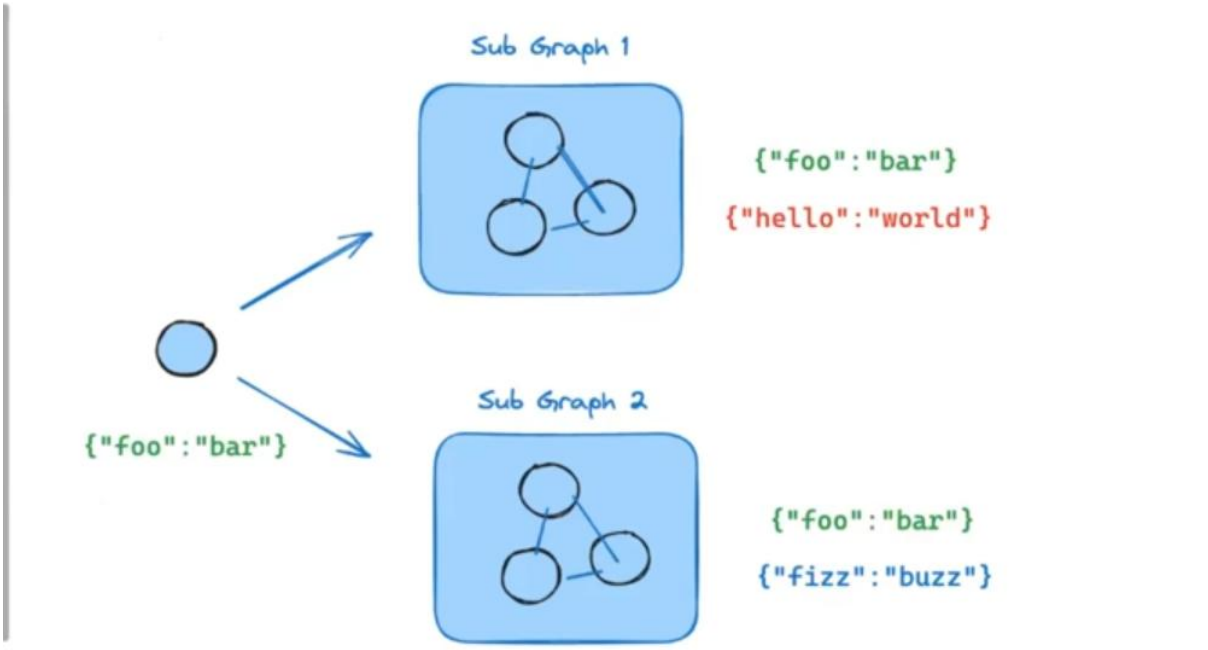


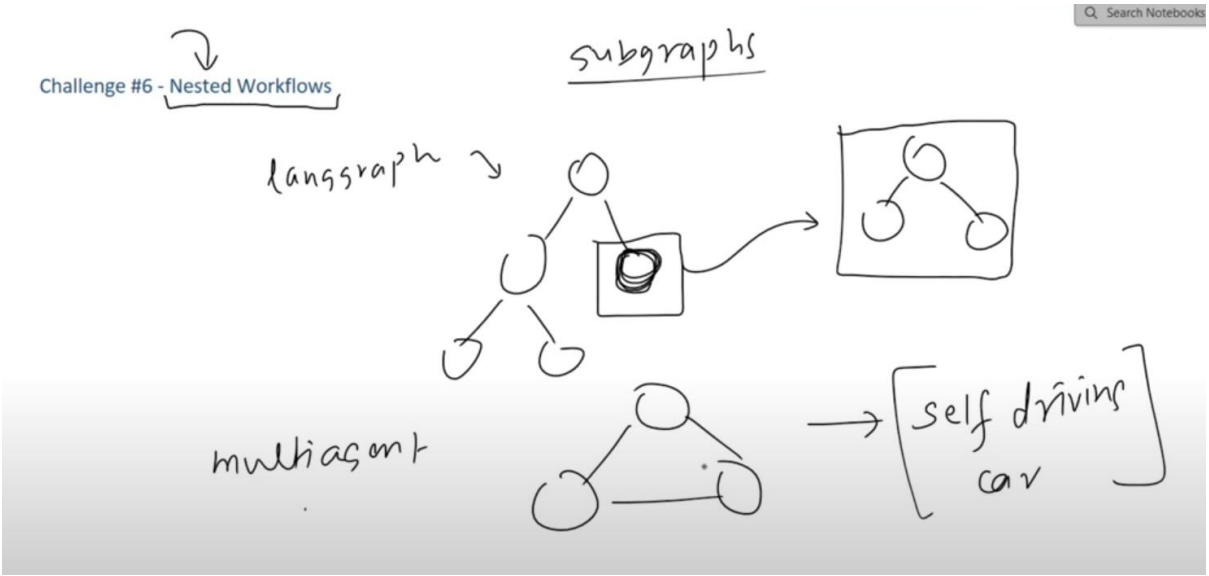
Challenge #5 - Human-in the loop

Challenge #6 - Nested Workflows

Subgraphs

A subgraph is a graph that is used as a node in another graph — this is the concept of encapsulation applied to LangGraph. Subgraphs allow you to build complex systems with multiple components that are themselves graphs.





Challenge #7 - Observability

Observability refers to how easily you can monitor, debug, and understand what your workflow is doing at runtime.

Conclusion

03 July 2025 08:00

1. What is LangGraph?

LangGraph is an orchestration framework that enables you to build **stateful**, **multi-step**, and **event-driven** workflows using large language models (LLMs). It's ideal for designing both **single-agent** and **multi-agent** agentic AI applications.

Think of LangGraph as a **flowchart engine for LLMs** — you define the steps (nodes), how they're connected (edges), and the logic that governs the transitions. LangGraph takes care of **state management**, **conditional branching**, **looping**, **pausing/resuming**, and **fault recovery** — features essential for building robust, production-grade AI systems.

2. When to Use What?

- Use **LangChain** when you're building **simple, linear workflows** — like a prompt chain, summarizer, or a basic retrieval system.
- Use **LangGraph** when your use case involves **complex, non-linear workflows** that need:
  - Conditional paths
  - Loops
  - Human-in-the-loop steps
  - Multi-agent coordination
  - Asynchronous or event-driven execution

3. Should We Still Use LangChain?

Yes. LangGraph is built **on top of LangChain** — it doesn't replace it.

You'll still use **LangChain components** like:

- `ChatOpenAI` (LLMs)
- `PromptTemplate`
- `Retrievers`
- `DocumentLoaders`
- `Tools`, etc.

LangGraph handles **workflow orchestration**, while LangChain provides the **building blocks** for each step in that workflow.